

```

timescale 1ns / 1ps
`default_nettype none
`include "parameters.vh"
`include "reg_file.v"
`include "imm_gen.v"
`include "control_unit.v"

module ID (
    input wire clk,
    input wire rst_n,
    input wire [31:0] i_Instr,          // Fetched instruction
    input wire signed [31:0] i_write_data, // Data to write to the register file
    input wire [31:0] i_Pc,            // PC of the current instruction
    input wire i_wr,                   // Write enable
    input wire [4:0] i_Rd,             // Destination register
    input wire flush, i_Boj,

    output wire [2:0] o_Func3E,        // func3 field
    output wire [4:0] o_AmoFunc5E,    // funct5 for AMO
    output wire o_IsAmoE,              // is this an AMO instruction
    output wire [31:0] o_Rd1E,         // rs1 data
    output wire [31:0] o_Rd2E,         // rs2 data
    output wire [31:0] o_ImmE,         // Immediate value
    output wire [31:0] o_PcE,          // PC for the next stage

    output wire o_RegSrcE,
    output wire [4:0] o_RdE,
    output wire o_Sel1E,
    output wire o_Sel2E,
    output wire [3:0] o_ALUCtrlE,      // ALU control signals
    output wire o_LoadE,
    output wire o_BranchE,
    output wire o_JalE,
    output wire o_JalrE,
    output wire o_MemSrcE,
    output wire [1:0] o_ResultSrcE,
    output wire [6:0] is_Opcode,

    output wire [4:0] a1, a2,
    output wire [4:0] is_Rs1, is_Rs2
);

// Internal signals
wire [2:0] is_Func3;
wire is_Re;
wire [3:0] o_alu_ctrl;

```

```

wire [31:0] o_read_data1;
wire [31:0] o_read_data2;
wire [4:0] is_Rd;
wire [31:0] imm_data;
wire [4:0] funct5 = i_Instr[31:27]; // upper 5 bits of funct7 field


reg [31:0] Rs1_reg, Rs2_reg;
reg [2:0] Func3_reg;
reg [4:0] Rd_reg;
reg [3:0] AluCtrl_reg;
reg [31:0] PC_reg;
reg [4:0] a1_reg, a2_reg;
reg [31:0] imm_reg;
reg [4:0] AmoFunct5_reg;
reg IsAmo_reg;


assign is_Opcode = i_Instr[6:0];
assign is_Func3 = i_Instr[14:12];
assign is_Rs1 = i_Instr[19:15];
assign is_Rs2 = i_Instr[24:20];
assign is_Rd = i_Instr[11:7];
assign is_Re = ~(is_Opcode == `J) | (is_Opcode == `U) | (is_Opcode == `UPC));


// Submodules
reg_file reg_file_inst (
    .clk(clk),
    .rst_n(rst_n),
    .i_re(is_Re),
    .i_wr(i_wr),
    .i_rs1(is_Rs1),
    .i_rs2(is_Rs2),
    .i_rd(i_Rd),
    .i_write_data(i_write_data),
    .o_read_data1(o_read_data1),
    .o_read_data2(o_read_data2)
);


imm_gen imm_gen_inst (
    .clk(clk),
    .rst_n(rst_n),
    .i_Instr(i_Instr),
    .imm_data(imm_data)
);


control_unit control_unit_inst (
    .clk(clk),

```

```

.rst_n(rst_n),
.i_Instr(i_Instr),
.o_RegSrcE(o_RegSrcE),
.o_Sel1E(o_Sel1E),
.o_Sel2E(o_Sel2E),
.o_alu_ctrl(o_alu_ctrl),
.o_LoadE(o_LoadE),
.o_BranchE(o_BranchE),
.o_JalE(o_JalE),
.o_JalrE(o_JalrE),
.o_MemSrcE(o_MemSrcE),
.o_ResultSrcE(o_ResultSrcE)
);

// Register logic
always @(posedge clk or negedge rst_n) begin
    if (~rst_n || flush || i_Boj) begin
        Func3_reg      <= 3'b0;
        AluCtrl_reg    <= 4'b0;
        Rs1_reg        <= 32'b0;
        Rs2_reg        <= 32'b0;
        PC_reg         <= 32'b0;
        Rd_reg         <= 5'b0;
        a1_reg         <= 5'b0;
        a2_reg         <= 5'b0;
        imm_reg        <= 32'b0;
        AmoFunct5_reg  <= 5'b0;
        IsAmo_reg      <= 1'b0;
    end else begin
        Func3_reg      <= is_Func3;
        AluCtrl_reg    <= o_alu_ctrl;
        Rs1_reg        <= o_read_data1;
        Rs2_reg        <= o_read_data2;
        PC_reg         <= i_Pc;
        Rd_reg         <= is_Rd;
        a1_reg         <= is_Rs1;
        a2_reg         <= is_Rs2;
        imm_reg        <= imm_data;
        AmoFunct5_reg  <= funct5;
        IsAmo_reg      <= (is_Opcode == `AMO);
    end
end

// Output assignments
assign o_Func3E      = Func3_reg;
assign o_ALUCtrlE    = AluCtrl_reg;

```

```
assign o_PcE      = PC_reg;
assign o_Rd1E     = Rs1_reg;
assign o_Rd2E     = Rs2_reg;
assign o_RdE      = Rd_reg;
assign a1         = a1_reg;
assign a2         = a2_reg;
assign o_ImmE     = imm_reg;
assign o_AmoFunct5E = AmoFunct5_reg;
assign o_IsAmoE   = IsAmo_reg;
```

```
endmodule
```